The Bioinformatics Toolbox

A Practical Tutorial on Shell, R, Git, Cloud Pipelines, and AI-Assisted Development

Zhenguo Zhang

Compiled from a series of LinkedIn technical posts

Contents

1 Preface **2**

2 Part I: Shell and Command-Line Productivity **2**
2.1 Chapter 1: iTerm2 Shell Integration — Seamless File Transfers . . . 2
2.2 Chapter 2: Quoting Rules — Shell vs. R . . . 3
2.3 Chapter 3: grep Exit Codes in Strict Bash Scripts . . . 4

3 Part II: R for Data Analysis and Visualization **6**
3.1 Chapter 4: ggplot2 Tip — Fix Tiny Legend Keys with `override.aes` . . . 6
3.2 Chapter 5: ggplot2 Tip — Position the Legend Inside the Plot . . . 8
3.3 Chapter 6: R — Adding Patterns to ggplot2 with `ggpattern` . . . 9

4 Part III: R Shiny — Interactive Apps and Deployment **10**
4.1 Chapter 7: Build an AI Chatbot in R with `shinychat` and `ellmer` . . . 10
4.2 Chapter 8: Environment Variables in R Shiny Server Containers . . . 12

5 Part IV: Version Control and Web Development **13**
5.1 Chapter 9: Git Worktree — Working on Multiple Branches Simultaneously . . . 13
5.2 Chapter 10: Modifying the blogdown (Hugo) Theme Safely . . . 15

6 Part V: Bioinformatics Pipelines and Cloud Infrastructure **16**
6.1 Chapter 11: Nextflow AWS Options — Where They Take Effect . . . 16
6.2 Chapter 12: Building a Searchable PDF Database with `pdf.db.builder` . . . 17

7 Part VI: Algorithms for String Matching **18**
7.1 Chapter 13: The Z-Algorithm for Linear-Time String Matching . . . 18

8 Part VII: AI-Assisted Development for Bioinformaticians **20**
8.1 Chapter 14: Making AI Agents Smarter with Skills . . . 20
8.2 Chapter 15: Creating Custom Agent Skills with the Gemini CLI . . . 21
8.3 Guardrails . . . 22

9 Part VIII: Reading the Literature with AI **25**
9.1 Chapter 18: Paper Summaries — Two Worked Examples and Critical Reading . . . 25

10 Appendix A: Quick Reference Cards **27**
10.1 A.1 Shell and Bash . . . 27
10.2 A.2 R and ggplot2 . . . 27
10.3 A.3 Shiny and containers . . . 27
10.4 A.4 Git worktree . . . 28
10.5 A.5 Nextflow AWS mapping . . . 28
10.6 A.6 Z-algorithm . . . 28

10.7 A.7 AI agent skills 28

11 Appendix B: Further Reading and Resources **28**
11.1 Original posts this tutorial is based on 28
11.2 Blog articles (full versions) 29
11.3 Repositories 29
11.4 Official documentation 29

1 Preface

This tutorial compiles and expands a series of technical posts originally published on LinkedIn. The material targets **bioinformaticians and computational biologists** who want to become more productive with the day-to-day tools of the trade: the command line, R and ggplot2, Shiny applications, version control, cloud pipelines, string-matching algorithms, and modern AI-assisted development.

Each chapter is self-contained and follows the same teaching structure:

- **Learning objectives** — what you should be able to do afterwards.
- **Background** — why the topic matters in a bioinformatics context.
- **Step-by-step explanation** with runnable code examples.
- **Common pitfalls** — the mistakes that cost people hours.
- **Exercises** — short tasks to consolidate the material.

Working through the chapters in order gives you a coherent tour of the "bioinformatics toolbox". Dipping into individual chapters works too, since every chapter stands alone.

---

2 Part I: Shell and Command-Line Productivity

2.1 Chapter 1: iTerm2 Shell Integration — Seamless File Transfers

Learning objectives

- Install iTerm2 shell integration on local and remote machines.
- Configure the critical `iterm2_hostname` variable.
- Upload and download files between your Mac and a remote server without typing a single `scp` command.

2.1.1 1.1 Background

Most bioinformatics work happens over SSH: you log into a cluster, inspect files with `ls`, run pipelines, and inspect logs. The recurring friction point is moving files between your laptop and the server. Classically this requires remembering `scp`/`rsync` syntax, or keeping an SFTP client open.

iTerm2 (a powerful terminal emulator for macOS) solves this with **shell integration**: a small script installed into your remote shell that lets iTerm2 know *where* you are and *what* is visible on screen. With that link in place, file transfers become mouse/keyboard gestures inside the terminal itself.

2.1.2 1.2 Installing shell integration

1. Open an SSH session to your remote server inside an iTerm2 window.
2. From the menu bar choose **iTerm2 -> Install Shell Integration**.
3. Follow the on-screen instructions (the installer appends a snippet to `~/.bashrc` or `~/.zshrc` on the remote).

2.1.3 1.3 The critical step: `iterm2_hostname`

For *file transfers* to work, iTerm2 must know the network name or IP of the remote machine. Set the `iterm2_hostname` environment variable in the remote shell profile:

```
# Add to ~/.bashrc or ~/.zshrc on the remote server
export iterm2_hostname=your-server-hostname
```

Find the right value on the remote server:

```
hostname        # short name; try this first
hostname -i     # fallback: the IP address
```

Without `iterm2_hostname`, iTerm2 cannot decide where files should go, and upload/download will silently not work. This is the single most common setup failure.

2.1.4 1.4 Downloading files from the remote server

Once configured, downloading is a click:

- **Right-click any filename** shown in terminal output (for example from `ls` results) and choose **Download with scp**.
- Alternatively, hold the **Cmd** key and click the filename.

Any file path visible on screen can be grabbed this way — no `scp` command construction required.

2.1.5 1.5 Uploading files to the remote server

- Hold the **Option** key and drag files from Finder directly into the iTerm2 window.
- Files are uploaded via `scp` automatically to your **current working directory** on the remote server.

2.1.6 1.6 Why this matters and common pitfalls

Shell integration turns file management in SSH sessions from a multi-step command process into single gestures. It is a genuine time-saver for daily cluster work.

Pitfalls

1. `iterm2_hostname` not set — transfers appear to do nothing.
2. The hostname is not resolvable from your Mac (DNS/`/etc/hosts`); use the IP address instead.
3. Shell integration script missing on the remote (forgot to install it, or a fresh login shell does not source the profile — use a login shell or add the snippet to `.profile`).

Exercises

1. Install shell integration on a server you use regularly and transfer one small file in each direction.
2. Remove `iterm2_hostname` from your profile and explain, based on Section 1.3, what breaks and why.
3. In which file should `iterm2_hostname` be defined so it survives every login? What if your shell is `zsh`?

2.2 Chapter 2: Quoting Rules — Shell vs. R

Learning objectives

- Explain how the shell expands double-quoted versus single-quoted strings.
- Apply the "double backslash" rule for R regular expressions.
- Pass command-line arguments safely between the shell and R.

2.2.1 2.1 Background

Bioinformatics workflows constantly mix languages: you call `Rscript` from a Bash pipeline, pass file paths and regex patterns as arguments, and read environment variables. Quoting bugs are among the most frustrating — a string is silently mangled before your R code even starts.

2.2.2 2.2 The shell: expansion rules

In Bash, the two quote characters have *different* jobs:

- **Double quotes**: the shell still performs *variable expansion* (`$VAR`), *command substitution* (`$(cmd)` and backticks), and *arithmetic expansion* (`$((...))`). The result is a single word (no word splitting).
- **Single quotes**: everything is fully **literal**. No expansion of any kind happens.

```
name="alice"
echo "hello $name"    # hello alice  (variable expanded)
echo 'hello $name'    # hello $name  (literally printed)

echo "today: $(date +%F)"    # today: 2026-08-15 (command run)
echo 'today: $(date +%F)'    # today: $(date +%F) (verbatim)
```

2.2.3 2.3 R: a different set of rules

In R, both `"` and `'` simply delimit strings — there is **no variable expansion inside either**. The real differences are:

- Convenience of nesting: `'it\\'s'` vs `"it's"`.
- The escaping of backslashes, which matters enormously for regular expressions.

The double-backslash rule. R's string parser already consumes one level of backslashes, so to give the regex engine a `\.` (escaped dot = literal dot) you must write `\\.` in your R source:

```
# A literal dot in a regex needs TWO backslashes in R source code:
grep("a\\.b", c("a.b", "axb"))    # matches only "a.b"

# ONE backslash means: dot = any single character (regex wildcard):
grep("a.b", c("a.b", "axb"))      # matches BOTH
```

The pattern `a.b` matches `a.b` and `axb` because `.` is a wildcard; `a\.b` matches only the literal string `a.b`.

2.2.4 2.4 Bringing the two together

A common pattern is passing a pattern from the shell into R:

```
# In Bash: single quotes protect the $ and backslashes from the shell
Rscript -e 'grep("a\\.b", readLines("genes.txt"))'
```

If you used double quotes here, the shell would try to expand `$` and interpret backslashes before R ever sees the string.

Common pitfalls

1. Double quotes in the shell triggering unwanted variable expansion of R code containing `$` (e.g. `$HOME`, tidyverse pronouns).
2. Forgetting the double backslash in R regexes, so `.` becomes a wildcard.
3. File paths containing spaces or special characters being split into multiple words — quote them.

Exercises

1. Predict the output of `echo "a $HOME b"` and `echo 'a $HOME b'`.
2. Why does `grep("a.b", "axb")` in R return a match? Write the pattern that matches only the literal dot.
3. Write an `Rscript -e` one-liner invoked from Bash that greps a file for the literal pattern `chr1:1000-2000` (note the colon and dash) and explain which quoting protects what.

2.3 Chapter 3: grep Exit Codes in Strict Bash Scripts

Learning objectives

- Interpret the Unix exit-code convention and `grep`'s specific codes.
- Explain why `grep` kills scripts running with `set -e` and `set -o pipefail`.
- Apply four practical fixes and choose the right one.

2.3.1 3.1 Background

Modern bioinformatics pipelines are glued together with Bash. To make scripts fail fast instead of silently producing garbage, developers turn on strict mode: `set -e` (exit on any failing command) and `set -o pipefail` (a pipeline fails if *any* of its members fails). These settings interact badly with one very common tool: `grep`.

2.3.2 3.2 The core issue: grep uses exit codes for search outcome

Unix tools conventionally return `0` on success and non-zero on error. `grep` follows a *different* contract, because "no match" is a normal search outcome:

Exit code	Meaning
`0`	Matching lines were found
`1`	**No matching lines found** (a completely normal outcome!)
`2` or higher	A real error (bad syntax, unreadable file)

2.3.3 3.3 The consequences

With `set -e`:

```
set -e
grep "CRITICAL" app.log      # no match -> exit 1 -> script aborts here
echo "still running"         # NEVER printed
```

Bash treats `grep`'s exit code `1` as a command failure and aborts the script instantly, even though the "failure" was just an empty search.

With `set -o pipefail`:

```
set -e
set -o pipefail
n=$(cat app.log | grep "CRITICAL" | wc -l)   # grep's 1 propagates
echo "count: $n"                             # NEVER reached
```

`pipefail` makes the pipeline exit with the *rightmost non-zero* status of its members, so `grep`'s `1` becomes the pipeline status and `set -e` kills the script before the output variable is assigned.

2.3.4 3.4 Four practical solutions

Solution 1 — `grep ... || true`

```
set -e
grep "CRITICAL" app.log || true
echo "continues"    # always runs
```

Quick and readable, but note it also masks real exit code `2` errors — you will not notice a missing file.

Solution 2 — conditional testing with `if` / `$?`

```
set -e
if grep -q "CRITICAL" app.log; then
    echo "CRITICAL found"
else
    echo "no CRITICAL lines"   # cleanly handles code 1
fi

# Or capture the status explicitly
```

```bash
grep -q "CRITICAL" app.log
status=$?
case $status in
  0) echo "found" ;;
  1) echo "not found" ;;
  *) echo "real error (code $status)" ;;
esac
```

This is the cleanest approach: you can distinguish "no match" from "real error" and handle each appropriately.

Solution 3 — `awk` pipeline filtering

```bash
set -e
set -o pipefail
n=$(cat app.log | awk '/CRITICAL/' | wc -l)
echo "count: $n"
```

`awk` returns `0` even when zero lines match (it only fails on genuine syntax/file errors), so `pipefail` stays happy while real errors still surface.

Solution 4 — temporarily toggle strict mode

```bash
set -e
set +e
grep "CRITICAL" app.log
set -e
```

Use sparingly: it is easy to forget to re-enable strict mode.

2.3.5 3.5 Choosing a solution

Approach	Masks code 2?	Handles "not found" explicitly?	Readability
`|| true`	yes	no	high
`if` / `$?`	no	yes	high
`awk` filter	no	n/a (returns 0)	medium
`set +e` toggle	no	no	low

Recommendation: use `if grep ...` when the search result changes program logic; use `awk` in pipelines when you only need counts; reserve `|| true` for cases where you genuinely do not care about the outcome.

Exercises

1. Write a strict-mode script that reports "CRITICAL found" or "no CRITICAL lines" without ever aborting.
2. Explain why `cat app.log | grep CRITICAL | wc -l` fails under `pipefail` but the `awk` equivalent does not.
3. Your pipeline needs to *fail* when the log file is missing, but *succeed* when the pattern is absent. Which solution fits and why?

---

3 Part II: R for Data Analysis and Visualization

3.1 Chapter 4: ggplot2 Tip — Fix Tiny Legend Keys with `override.aes`

Learning objectives

- Diagnose why legend keys shrink when data point sizes are reduced.
- Use `override.aes` inside `guide_legend()` to size legend symbols independently of the data.
- Apply the same trick to other aesthetic overrides.

3.1.1 4.1 The problem

In bioinformatics figures, overplotting is a constant enemy: thousands of points plotted on top of each other become an unreadable blob. The standard remedy is to shrink the point size. But there is an annoying side effect — **legend keys inherit the point size** of the data, so your legend becomes unreadable too:

```
library(ggplot2)

# Points sized by a continuous variable: legend keys get tiny too
p <- ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl), size = disp)) +
  geom_point(alpha = 0.7)
print(p)
```

3.1.2 4.2 The solution: `override.aes`

The legend is controlled by the `guides()` function. For a legend driven by the `size` aesthetic, override the size only for the legend keys:

```
p + guides(size = guide_legend(override.aes = list(size = 3)))
```

You can override any aesthetic shown in the legend this way — `color`, `shape`, `fill`, `alpha`, and so on:

```
p + guides(
  color = guide_legend(override.aes = list(size = 3, alpha = 1)),
  size  = guide_legend(override.aes = list(size = 3))
)
```

3.1.3 4.3 Full worked example

```
library(ggplot2)

p <- ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl), size = disp)) +
  geom_point(alpha = 0.6) +
  guides(
    color = guide_legend(override.aes = list(size = 3, alpha = 1)),
    size  = guide_legend(override.aes = list(size = 3, alpha = 1))
  ) +
  labs(title = "Legend keys independent of data point size")
print(p)

ggsave("legend_override.png", width = 7, height = 5, dpi = 300)
```

3.1.4 4.4 Common pitfalls

1. Trying to fix the legend by making the data points bigger — this reintroduces overplotting.
2. Forgetting that discrete legends (e.g. `color`) and continuous ones (e.g. `size`) are separate legends and need separate `override.aes`.
3. Setting `override.aes` but leaving `alpha` at its tiny value, so the legend key is big but nearly invisible.

Exercises

1. Reproduce the tiny-legend problem with `mtcars` and fix it with `override.aes`.
2. Use `override.aes` to change the legend key *shape* (hint: `shape` is an aesthetic too) for a plot where points use multiple shapes.
3. Why does the legend key shrink when you reduce `size` in `geom_point()`? Explain in terms of how legends inherit aesthetics.

3.2 Chapter 5: ggplot2 Tip — Position the Legend Inside the Plot

Learning objectives

- Move a ggplot2 legend inside the plotting area with relative coordinates.
- Anchor the legend precisely with `legend.justification`.
- Clean up the legend box for an unobtrusive look.

3.2.1 5.1 Background

Figures in papers and posters have limited space. A legend parked outside the plot area consumes a large fraction of the canvas — the "data-ink ratio" (ink spent on non-data elements) suffers. Moving the legend inside the plot area is a classic fix, and ggplot2 makes it surprisingly easy.

3.2.2 5.2 The core technique

`legend.position` accepts a numeric vector of length 2 with values in [0, 1], interpreted as fractions of the plot area:

- `c(0, 0)` — bottom-left corner
- `c(1, 1)` — top-right corner
- `c(0.85, 0.3)` — right side, about 30% up

`legend.justification` controls *which corner of the legend box* is anchored at that position.

```
library(ggplot2)

p <- ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point(size = 3) +
  theme(
    legend.position = c(0.85, 0.8),          # inside, top-right area
    legend.justification = c(0.5, 0.5),      # anchor legend center
    legend.background = element_blank(),     # no gray box
    legend.key = element_blank()             # no key background
  )
print(p)
```

3.2.3 5.3 Fine-tuning

- If the legend overlaps data points, nudge the position or add `legend.background = element_rect(fill = "white", color = "gray50")` for a semi-opaque backdrop.
- Use `legend.title = element_blank()` to drop titles for a cleaner look.
- On small plots, relative coordinates can push the legend off the edge — check the rendered output and adjust.

```
p + theme(
  legend.position = c(0.15, 0.85),            # top-left
  legend.justification = c(0, 0),             # anchor top-left corner
  legend.background = element_rect(
    fill = alpha("white", 0.8), color = "gray50"
  )
)
```

3.2.4 5.4 Common pitfalls

1. Using absolute coordinates (e.g. `c(100, 50)`) — `legend.position` wants **relative** values in [0, 1].
2. Forgetting `legend.justification`, so the legend box "walks" away from where you think you placed it.
3. The legend background box clashing with the data — blank it or use a translucent fill.

Exercises

1. Put a legend in the bottom-right corner of a scatter plot, anchored by its bottom-right corner, with no background box.

2. Explain the difference between `legend.position` and `legend.justification` with a concrete example.
3. When would a translucent legend background be better than `element_blank()`? Find a plot in your own work that would benefit.

3.3 Chapter 6: R — Adding Patterns to ggplot2 with ggpattern

Learning objectives

- Install and load `ggpattern`.
- Add patterns (stripes, crosshatches) to ggplot2 geoms.
- Customize pattern density, spacing, and angle, and fix legend keys.

3.3.1 6.1 Background and motivation

Color is the default way to distinguish groups, but color-only figures have real limitations:

- **Accessibility**: roughly 8% of men have some form of color vision deficiency; patterns keep charts readable.
- **Extra dimensions**: patterns let you encode another variable without adding more colors.
- **Aesthetics and print**: patterns produce professional, print-ready graphics (e.g. for journals with gray-scale printing).

3.3.2 6.2 Installation and first plot

```r
install.packages("ggpattern")   # once
library(ggplot2)
library(ggpattern)

df <- data.frame(
  gene = c("TP53", "BRCA1", "EGFR", "MYC"),
  value = c(12, 8, 15, 6),
  group = factor(c("oncogene", "oncogene", "oncogene", "oncogene"))
)

# Simple patterned bars
ggplot(df, aes(x = gene, y = value)) +
  geom_col_pattern(
    pattern = "stripe",            # stripe, crosshatch, circle, ...
    fill    = "lightblue",
    colour  = "black"
  ) +
  theme_minimal()
```

3.3.3 6.3 Customizing pattern appearance

Key arguments: `pattern_density` (0-1, spacing between elements), `pattern_spacing` (gap size in cm), `pattern_angle` (degrees), `pattern_fill` / `pattern_colour`:

```r
ggplot(df, aes(x = gene, y = value, fill = group)) +
  geom_col_pattern(
    pattern         = "crosshatch",
    pattern_density  = 0.3,
    pattern_spacing  = 0.05,
    pattern_angle    = 45,
    pattern_fill     = "white",
    colour           = "black"
  )
```

3.3.4 6.4 Patterns for grouped comparisons

```
df2 <- data.frame(
  tissue = rep(c("tumor", "normal"), each = 4),
  gene   = rep(c("TP53", "BRCA1", "EGFR", "MYC"), 2),
  value  = c(12, 8, 15, 6, 3, 5, 9, 2)
)

ggplot(df2, aes(x = gene, y = value, fill = tissue)) +
  geom_col_pattern(
    aes(pattern = tissue),       # pattern also encodes the group
    pattern_density = 0.35,
    pattern_spacing = 0.04
  ) +
  scale_fill_manual(values = c("tumor" = "tomato", "normal" = "skyblue")) +
  theme_minimal()
```

3.3.5 6.5 Legend keys

Patterned legends can look cramped; enlarge the keys with the `override.aes` trick from Chapter 4, and remember the pattern aesthetics follow the same legend machinery:

```
... +
  guides(
    fill    = guide_legend(override.aes = list(pattern_spacing = 0.06)),
    pattern = guide_legend(override.aes = list(pattern_spacing = 0.06))
  )
```

3.3.6 6.6 Common pitfalls

1. Installing `ggpattern` but forgetting to load it with `library(ggpattern)`.
2. Using pattern geoms that require a `fill` aesthetic to be mapped — patterns apply per fill group.
3. Legend keys too dense to see — increase `pattern_spacing` via `override.aes`.

Exercises

1. Reproduce the grouped bar chart in 6.4 and verify the legend is readable.
2. Try `pattern = "circle"` and `pattern = "wave"`; which two pattern arguments control their appearance?
3. Explain how patterns make a figure more accessible to colorblind readers, and when you would still prefer color alone.

---

4 Part III: R Shiny — Interactive Apps and Deployment

4.1 Chapter 7: Build an AI Chatbot in R with `shinychat` and `ellmer`

Learning objectives

- Install `shinychat` and `ellmer` and set up credentials.
- Build a streaming chat interface with two function calls.
- Connect multiple LLM providers and extend the chatbot with R functions.

4.1.1 7.1 Background

Interfaces between bioinformaticians and LLMs are increasingly part of daily work. Instead of switching to a browser chat, you can embed a chat window directly inside an R Shiny app — perfect for wrapping your analysis tools with a natural-language front end.

Two Posit packages do the heavy lifting:

- **shinychat** — a polished, streaming chat UI module for Shiny.
- **ellmer** — a unified interface to many LLM providers (OpenAI, Anthropic, Google Gemini, AWS Bedrock, and more).

4.1.2 7.2 Setup

```
install.packages(c("shinychat", "ellmer"))
```

You need API credentials, normally stored as environment variables (see Chapter 8 for a container gotcha!):

```
export OPENAI_API_KEY=sk-...
export ANTHROPIC_API_KEY=sk-ant-...
export GEMINI_API_KEY=...
```

4.1.3 7.3 A minimal chatbot app

```
library(shiny)
library(shinychat)
library(ellmer)

ui <- fluidPage(
  titlePanel("Bioinformatics Chat Assistant"),
  chat_mod_ui("chat")           # the chat UI module
)

server <- function(input, output, session) {
  chat <- chat_openai(
    system_prompt = paste(
      "You are a helpful bioinformatics assistant.",
      "Answer concisely and cite tools when relevant."
    )
  )
  chat_mod_server("chat", chat)   # wire UI to the chat client
}

shinyApp(ui, server)
```

Run the app; you get a streaming chat interface with zero CSS or JavaScript. Switch providers by swapping the chat constructor:

```
# Anthropic
chat <- chat_anthropic(model = "claude-3-5-sonnet-latest",
                       system_prompt = "...")

# Google Gemini
chat <- chat_google_gemini(system_prompt = "...")

# AWS Bedrock
chat <- chat_bedrock(model = "...", system_prompt = "...")
```

4.1.4 7.4 Making the chatbot useful: tools and extensibility

The real power comes from letting the model call R code. **ellmer** supports tool registration, so the chatbot can, for example, run functions that summarize a data frame, fetch a file, or draw a plot:

```
library(ellmer)

# A simple R function the model may call
```

```r
read_bed_summary <- function(path) {
  x <- read.table(path, header = FALSE, nrows = 3)
  paste(capture.output(x), collapse = "\n")
}

chat <- chat_openai() |>
  register_tool(
    fun = read_bed_summary,
    description = "Read the first three lines of a BED file and return them",
    arguments = list(path = tool_arg(type = "string",
                                     description = "Path to BED file"))
  )
```

Note: tool registration API details may vary by `ellmer` version — check `help("register_tool")` for the current signature.

4.1.5 7.5 Common pitfalls

1. Missing API key environment variables — the chat constructor throws or silently fails.
2. Forgetting `chat_mod_server` — the UI renders but nothing answers.
3. Passing a fresh chat client per session (fine) versus reusing a conversation across users (state leaks between users in production).

Exercises

1. Build the minimal app and chat with it locally.
2. Add a second provider behind a `selectInput` so users can pick the model.
3. Register a tool that draws a ggplot from user-described data and returns the plot file path. Test it end-to-end.

4.2 Chapter 8: Environment Variables in R Shiny Server Containers

Learning objectives

- Diagnose why Docker `ENV` variables disappear inside Shiny Server.
- Apply two clean fixes: `~/.profile` and `~/.Renviron`.

4.2.1 8.1 The problem

You containerize an R Shiny app with Shiny Server:

```
FROM rocker/shiny:4.4
ENV DB_PASSWORD=secret
COPY app.R /srv/shiny-server/app.R
```

Inside the app, `Sys.getenv("DB_PASSWORD")` returns `""` — even though `docker run -e DB_PASSWORD=...` also seems to pass the value. The variable is silently gone when R starts.

4.2.2 8.2 The root cause: it is not a Docker bug

Shiny Server launches R worker processes with a command like:

```
su shiny --login --preserve-environment -c "...Rscript..."
```

The `su` invocation asks for *both* `--login` (start a login shell, which **wipes** the environment for a clean shell) and `--preserve-environment` (keep the current environment). These two flags contradict each other, and **the login shell wins**: all Docker `ENV` variables are wiped before R even starts.

4.2.3 8.3 Fix 1 — the user shell profile (`~/.profile`)

Because the login shell *does* source the user's profile, exporting variables there restores them:

```
FROM rocker/shiny:4.4
ENV DB_PASSWORD=secret
RUN echo 'export DB_PASSWORD=secret' >> /home/shiny/.profile
```

4.2.4 8.4 Fix 2 — the R environment file (`~/.Renviron`)

R natively reads `~/.Renviron` at startup, in the syntax `KEY=value` (no `export`):

```
FROM rocker/shiny:4.4
ENV DB_PASSWORD=secret
RUN echo 'DB_PASSWORD=secret' >> /home/shiny/.Renviron \
    && chown shiny:shiny /home/shiny/.Renviron
```

The `chown` matters: if `~/.Renviron` belongs to `root`, the `shiny` user may not be able to read it, depending on permissions.

4.2.5 8.5 Verification

```
docker build -t shiny-app .
docker run --rm -p 3838:3838 shiny-app
```

Then, inside the app or a quick check:

```
# Add temporarily to app.R to debug
print(Sys.getenv("DB_PASSWORD"))
```

4.2.6 8.6 Common pitfalls

1. Putting variables only in the `Dockerfile` `ENV` and expecting Shiny Server workers to inherit them.
2. Editing system-wide files (`/etc/environment`) instead of the `shiny` user's profile.
3. Forgetting `chown` on `~/.Renviron`.
4. Using `Sys.setenv()` inside `app.R` as a workaround — it works but duplicates secrets in code and does not survive worker restarts.

Exercises

1. Build the Dockerfile above, run the container, and confirm `Sys.getenv("DB_PASSWORD")` works with each fix.
2. Explain in one paragraph why `--login` and `--preserve-environment` cannot both be satisfied.
3. Which fix would you prefer for *secrets* (e.g. API keys) and why? (Hint: think about where the value lives in the image.)

---

5 Part IV: Version Control and Web Development

5.1 Chapter 9: Git Worktree — Working on Multiple Branches Simultaneously

Learning objectives

- Explain why `git stash` + switch and clone-again are suboptimal.
- Create, list, and remove worktrees.
- Use worktrees for hotfixes and parallel background tasks.

5.1.1 9.1 Background: the classic dilemma

You are deep in a feature branch with a dozen uncommitted files when a critical bug report arrives. Two classical reactions:

1. **Stash & switch**: `git stash`, checkout `main`, fix the bug, commit, switch back, `git stash pop`. It works, but it breaks your flow and context; stashes can even conflict after a long time.
2. **Clone again**: clone the repository into another folder. This duplicates the entire history — slow and disk-hungry.

5.1.2 9.2 The better way: `git worktree`

A worktree is a *second working directory* linked to the same repository, checked out on a *different branch*. All worktrees share a single `.git` database; only the working-tree files differ.

```
# Create a new worktree on a new branch (e.g. for a hotfix)
git worktree add ../myrepo-hotfix -b hotfix/fix-bug

# Create a worktree on an existing branch
git worktree add ../myrepo-release release/1.0

# List all worktrees
git worktree list

# Remove a worktree when done (after committing or discarding changes)
git worktree remove ../myrepo-hotfix

# Prune stale worktree metadata after deleting directories manually
git worktree prune
```

5.1.3 9.3 Why this is a game-changer

- **No stashing required**: keep your feature-branch work exactly as it is, and spin up a separate folder for the hotfix in seconds.
- **Minimal disk usage**: unlike multiple clones, worktrees share the object database; only the checked-out files are duplicated.
- **Parallel work**: run a long test suite in one worktree while continuing development in another.

5.1.4 9.4 A realistic bioinformatics workflow

```
# You are developing variant-calling features on branch 'feature/vqsr'
git worktree add ../pipeline-hotfix -b hotfix/vep-version
cd ../pipeline-hotfix
# fix the VEP version pin, commit, push
git commit -am "pin VEP 110"
git push origin hotfix/vep-version
cd .. && git worktree remove ../pipeline-hotfix
```

5.1.5 9.5 Common pitfalls

1. Trying to check out the *same* branch in two worktrees — Git refuses because the branch is already checked out elsewhere.
2. Removing a worktree with uncommitted changes — Git refuses; commit, stash, or use `--force` (with care).
3. Forgetting that the new worktree is a separate directory: your shell must `cd` into it; relative paths and IDE projects need updating.

Exercises

1. In a practice repo, create a worktree for a hotfix, commit a change, and remove it.
2. Explain why worktrees use less disk than a second clone.
3. What happens if you try `git worktree add` on a branch already checked out in another worktree? Try it and read the error.

5.2 Chapter 10: Modifying the blogdown (Hugo) Theme Safely

Learning objectives

- Understand Hugo's lookup order for layouts and assets.
- Customize a blogdown theme with local overrides.
- Use the submodule-forking strategy for extensive changes.

5.2.1 10.1 Background

blogdown is the R package for building websites with Hugo and R Markdown — a popular choice for lab websites and personal blogs in the R/biostatistics community. Themes are maintained upstream, and updating them is a double-edged sword: you get improvements, but your customizations can be silently overwritten or break on version incompatibilities.

5.2.2 10.2 Strategy 1 — local overrides with Hugo's lookup order

Hugo resolves templates using a lookup order: **project files take precedence over theme files**. If you copy a theme template into your project's `layouts/` directory (same path), your copy wins, while the theme keeps its original for future updates.

```
# Example: customize the single-page template
cp themes/my-theme/layouts/_default/single.html layouts/_default/single.html
# ... edit layouts/_default/single.html to your taste
```

The same applies to static assets (copy into `static/`) and shortcodes (copy into `layouts/shortcodes/`). Updating the theme no longer touches your overrides.

5.2.3 10.3 Strategy 2 — submodule forking for extensive changes

When you need deep, pervasive changes (redesigning the layout engine, adding features across many templates), the override approach becomes tedious. Instead, fork the theme:

1. Fork the theme repository on GitHub.
2. Add your fork as a git **submodule** of your site.
3. Make changes on your own branch; merge upstream updates into it.

```
git submodule add https://github.com/YOU/my-theme.git themes/my-theme
cd themes/my-theme
git checkout -b my-customizations
# ... make changes, commit, push to your fork
```

5.2.4 10.4 Choosing a strategy

Situation	Recommended strategy
One-off tweaks, occasional theme updates	Local overrides
Heavy, permanent customization	Submodule fork
Many collaborators with different needs	Fork + feature branches
You barely touch the theme	Neither — just update it

5.2.5 10.5 Common pitfalls

1. Editing files inside `themes/` directly — your changes vanish on the next theme update.
2. Overriding a layout but not the partials it calls, producing a half-customized page.
3. Forgetting hugo's cache after template changes — restart the server with `--disableFastRender` while iterating.

Exercises

1. Override the `single.html` template of your blogdown site with a trivial modification and verify it takes effect.

2. Explain how Hugo's lookup order prevents your local overrides from being clobbered by theme updates.
3. When would you choose the submodule fork over local overrides?

---

6 Part V: Bioinformatics Pipelines and Cloud Infrastructure

6.1 Chapter 11: Nextflow AWS Options — Where They Take Effect

Learning objectives

- Distinguish the two execution environments in an AWS Batch pipeline.
- Map `aws.client.*` and `aws.batch.*` options to the right layer.
- Tune S3 transfer settings where they actually apply.

6.1.1 11.1 Background: two environments, one pipeline

Nextflow pipelines running on AWS Batch move data between S3 and compute in **two completely different environments**:

1. **The head node** — the host running the Nextflow JVM. It publishes directories, uploads runner scripts, and gathers outputs.
2. **The EC2 execution instances** — each Batch task container. Here the container's `aws` S3 CLI or `s5cmd` stages files in and out.

An option like `aws.client.maxConnections` may do nothing for the container workload, because it configures a client that never runs on the EC2 instance. This is why tuning feels random.

6.1.2 11.2 The layer map

```
// nextflow.config (fragment)
aws {
  client {
    maxConnections   = 10   // head-node JVM client only
    multipartThreshold = "64 MB"
    connectionTimeout = "60s"
    storageClass     = "STANDARD_IA"  // translated to container too!
    s3Acl            = "private"      // translated to container too!
  }
  batch {
    // governs the in-container S3 CLI / s5cmd staging
    cpus = 2
    // ... container-side transfer behavior
  }
}
```

Option group	Layer	What it governs
`aws.client.*`	Head-node JVM client	publishing dirs, runner scripts, gathering outputs
`aws.batch.*`	EC2 execution instance	in-container `aws` CLI / `s5cmd` staging
Fusion FS	Filesystem layer	bypasses S3 CLI staging entirely

6.1.3 11.3 The critical caveat

Most `aws.client.*` options never propagate to the EC2 instances. Only a specific subset is translated via Nextflow's `S3BashLib` and takes effect inside the container S3 CLI — notably `storageClass` and `s3Acl`. If

you are trying to prevent *container* S3 network exhaustion, tune the layer that actually runs there (`aws.batch.*` / container-side settings), not `aws.client.*`.

6.1.4 11.4 Fusion: a different model entirely

The **Fusion File System** replaces S3 staging with a filesystem abstraction: S3 appears as local storage and the S3 CLI is bypassed altogether. With Fusion, the relevant tuning moves to Fusion/wave configuration (e.g. `fusion.exportStorage`, container image parameters) rather than S3 CLI options.

6.1.5 11.5 Practical workflow

1. Identify the bottleneck: head-node transfers or in-container staging?
2. Tune the corresponding layer; verify in the logs which client ran.
3. If S3 CLI staging is the bottleneck, consider Fusion.

> **Note**: the mapping above was researched with the help of an AI agent by digging into the Nextflow codebase. Verify against your Nextflow version and the official AWS Batch docs; config names evolve.

Exercises

1. Draw a diagram of the two environments and label which options govern each.
2. A colleague sets `aws.client.maxConnections = 50` to speed up container S3 transfers. Explain why this will not help.
3. When would switching to Fusion change which config file you edit?

6.2 Chapter 12: Building a Searchable PDF Database with `pdf.db.builder`

Learning objectives

- Describe the architecture: GROBID, SQLite FTS5, and a Shiny UI.
- Understand how incremental builds keep re-ingestion fast.
- Run the tool on your own PDF library.

6.2.1 12.1 Background: the haystack problem

Every researcher has a folder full of PDFs — papers, preprints, reports. Finding "that one paper about motif discovery in ATAC-seq peaks" usually means opening PDFs one by one. `pdf.db.builder` solves this by turning a local PDF library into a **searchable database** with a web interface.

6.2.2 12.2 Architecture

The tool (R + Python, AI-assisted development) has four moving parts:

1. **GROBID metadata extraction** — GROBID (a machine-learning document-parsing service, run via Docker) extracts bibliographic metadata from each PDF: title, authors, year, abstract, journal, DOI.
2. **SQLite + FTS5 storage** — metadata is ingested into a SQLite database with a full-text search (FTS5) index for lightning-fast keyword queries and ranking.
3. **Incremental ingest pipeline** — unchanged PDFs are skipped on re-runs, so updates are fast even for large libraries.
4. **Shiny web interface** — search metadata, select a paper, and open the original PDF in your system's default viewer.

6.2.3 12.3 Running it

```
# 1. Start the GROBID service (Docker)
docker run -d -p 8070:8070 lfoppiano/grobid:0.8.0

# 2. Clone and configure
git clone https://github.com/fortune9/pdf_db_builder
```

```
cd pdf_db_builder
# follow README: point the ingest script at your PDF folder

# 3. Ingest (first run builds the whole index; later runs are incremental)
Rscript ingest.R --pdf-dir ~/papers

# 4. Launch the search interface
Rscript run_app.R
# open http://localhost:3838 in your browser
```

6.2.4 12.4 Considerations and honest caveats

- The project is **early-stage and AI-assisted**: functional and ready to use, but with room for improvement (error handling, parsing edge cases, richer search features). Contributions are welcome.
- GROBID accuracy varies with PDF quality (scanned vs. digital, two-column layouts).
- FTS5 searches metadata fields — full-text of the body requires a different pipeline.

Exercises

1. Diagram the data flow: PDF -> GROBID -> SQLite FTS5 -> Shiny UI.
2. Why does the incremental design matter for a growing PDF library?
3. When would FTS5 metadata search fail to find a paper you know you have? Propose an enhancement.

---

7 Part VI: Algorithms for String Matching

7.1 Chapter 13: The Z-Algorithm for Linear-Time String Matching

Learning objectives

- Define the Z-array and compute it by hand for a small string.
- Implement the linear-time Z-algorithm.
- Use Z-values to find all occurrences of a pattern in a text.

7.1.1 13.1 Background

Finding patterns in sequences is core bioinformatics: motif discovery, k-mer matching, read alignment pre-filtering, primer design. The naive approach compares the pattern against every position of the text — `O(n x m)` in the worst case, where `n` is text length and `m` is pattern length. For genome-scale texts, that is far too slow.

The **Z-algorithm** solves exact pattern matching in `O(n + m)` time using a simple but powerful data structure: the Z-array.

7.1.2 13.2 The Z-array

For a string `S` of length `n`, define `Z[i]` as the length of the longest substring starting at position `i` that matches a **prefix** of `S`. Convention: `Z[0] = 0` (or `n`, depending on definition).

Worked example — `S = "aabcaabxaaaz"`:

i	compare S[i:] with S	Z[i]
0	(by convention)	0
1	a vs a, then b vs a -> stop	1
2	b vs a -> stop	0
3	c vs a -> stop	0
4	a=a, a=a, b vs c -> stop	2
5	a=a, x vs a -> stop	1
6	x vs a -> stop	0

i	compare S[i:] with S	Z[i]
7	a=a, a=a, a=b -> stop	3
8	a=a, z vs a -> stop	1
9	a=a, a=a, z vs b -> stop	2
10	a=a, z vs b -> stop	1
11	z vs a -> stop	0

So `Z = [0, 1, 0, 0, 2, 1, 0, 3, 1, 2, 1, 0]`.

7.1.3 13.3 The linear-time algorithm with the Z-box

A naive computation is `O(n^2)`. The trick is the **Z-box**: maintain the interval `[l, r]` such that `S[l..r]` is a prefix of `S` and `r` is as far right as possible.

- When `i <= r`, we are inside a known Z-box. Since `S[l..r]` matches the prefix, position `i` mirrors position `i - l`: use `Z[i - l]` but never extend past `r` without verifying: `Z[i] = min(r - i + 1, Z[i - l])`.
- Then extend by naive comparison while `S[Z[i]] == S[i + Z[i]]`.
- If the extension reaches past `r`, update `l = i, r = i + Z[i] - 1`.

Because each successful character comparison moves `r` right and `r` never moves left, the total work is `O(n)`.

```
def z_algorithm(s: str) -> list[int]:
    n = len(s)
    z = [0] * n
    l = r = 0
    for i in range(1, n):
        if i <= r:
            z[i] = min(r - i + 1, z[i - l])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1
        if i + z[i] - 1 > r:
            l, r = i, i + z[i] - 1
    return z
```

7.1.4 13.4 Pattern matching with the Z-array

To find all occurrences of pattern `P` (length `m`) in text `T` (length `n`):

1. Build `S = P + "$" + T` (a separator character that never appears in `P` or `T`).
2. Compute the Z-array of `S`.
3. Every position `i` with `Z[i] == m` is a match; the occurrence starts at text position `i - m - 1`.

```
def find_pattern(P: str, T: str) -> list[int]:
    s = P + "$" + T
    z = z_algorithm(s)
    m = len(P)
    return [i - m - 1 for i in range(m + 1, len(s)) if z[i] == m]

print(find_pattern("ACG", "TACGACGTACG"))   # [1, 4, 7]
```

Verify: `T = TACGACGTACG` — positions 1, 4, 7 indeed contain `ACG`.

7.1.5 13.5 Applications in bioinformatics

- **Exact motif/pattern search** in genomic sequences (linear time).
- **Read mapping pre-filtering** — exact seeds that are later extended with mismatches.
- Building blocks for more complex algorithms (e.g. computing longest common prefixes across suffixes, used in genome assembly scaffolding).

7.1.6 13.6 Common pitfalls

1. Off-by-one errors on the occurrence start position (`i - m - 1`, not `i - m`).
2. Forgetting the separator must not occur in either string.
3. Confusing `Z[i]` with the longest common prefix of suffixes — the definition is specifically *prefix of the whole string*.

Exercises

1. Compute the Z-array of `"ababca"` by hand and verify with the code.
2. Trace the Z-box update on `S = "aabcaabxaaaz"` for `i = 7`: why does the mirroring shortcut apply, and why is `Z[7]` capped at `r - 7 + 1`?
3. Prove/argue why the total time is linear even though the inner `while` loop can run many times across iterations.
4. Use `find_pattern` to locate all occurrences of a short motif in a genome FASTA (you may need to strip line breaks first).

---

8 Part VII: AI-Assisted Development for Bioinformaticians

8.1 Chapter 14: Making AI Agents Smarter with Skills

Learning objectives

- Explain what agent skills are and how they extend AI coding agents.
- Install pre-built skills from popular repositories.
- Judge when a skill is worth adding to your workflow.

8.1.1 14.1 Background

AI coding agents (Claude Code, GitHub Copilot, Cursor, Gemini CLI, Qwen Code, and others) are powerful out of the box, but they repeat the same struggles: they re-learn your conventions every session, they suggest generic solutions instead of your team's standards, and complex workflows have to be re-explained in every prompt.

Agent skills fix this. A skill is a **reusable instruction set** — typically a `SKILL.md` file plus optional scripts and references — that extends an agent with specialized knowledge and workflows. Skills are portable across platforms, so a skill written once can standardize team conventions everywhere.

8.1.2 14.2 What a skill looks like

A skill is a directory with a `SKILL.md` file containing:

- **Frontmatter**: `name` and `description` (the description tells the agent *when* to use the skill).
- **Instructions**: step-by-step guidance written in Markdown.
- **Resources**: additional files (scripts, reference documents) the skill can read.

The description matters most: the agent reads descriptions to decide which skill applies to the current task.

8.1.3 14.3 Installing pre-built skills

Several repositories publish battle-tested skills:

- **skills.sh** — 443K+ installs, a large catalog of community skills.
- **Vercel Labs** — web/frontend-focused skills.
- **Posit Dev** — R and data-science skills (e.g. R package development).

Install with the `npx skills` command-line tool:

```
# Install the r-package-development skill for Gemini CLI
npx skills add posit-dev/skills --skill r-package-development -a gemini-cli

# List installed skills
```

```
npx skills list

# Update skills
npx skills update
```

8.1.4 14.4 Why skills matter for bioinformatics teams

- **Standardize conventions**: a skill that encodes your lab's pipeline structure (Nextflow conventions, file layouts) makes every agent follow the same rules.
- **Share expertise**: senior members encode hard-won knowledge once; juniors and agents both benefit.
- **Codify workflows**: multi-step procedures (e.g. "annotate a VCF", "build an R package") become one prompt instead of twenty.

8.1.5 14.5 Common pitfalls

1. Installing too many skills — the agent spends context deciding which applies. Keep the set small and well-described.
2. Skills with vague `description` fields never get triggered.
3. Storing secrets inside skill files (e.g. API keys) — skills may be shared; keep them secret-free.

Exercises

1. Install one skill from skills.sh or Posit Dev and ask your agent to use it on a real task.
2. Read two `SKILL.md` files and critique their descriptions: would an agent know when to trigger them?
3. Sketch a skill that encodes your own lab's Nextflow project layout.

8.2 Chapter 15: Creating Custom Agent Skills with the Gemini CLI

Learning objectives

- Write a `SKILL.md` with effective frontmatter and instructions.
- Add resources (scripts, references) to a skill.
- Install, test, and iterate on a custom skill.

8.2.1 15.1 Background

When pre-built skills do not fit, write your own. Custom skills solve the "manual task repetition" problem: instead of pasting the same multi-step instructions into every prompt, you codify them once and the agent applies them consistently.

8.2.2 15.2 Where skills live

The exact locations depend on the agent. The general convention:

- **Project-level**: `.gemini/skills/<skill-name>/SKILL.md` (Gemini CLI) — travels with the repository.
- **User-level**: `~/.gemini/skills/<skill-name>/SKILL.md` — available everywhere on your machine.
- Claude Code: `~/.claude/skills/<skill-name>/SKILL.md`; Cursor, Codex, Cline and others follow similar conventions (check each tool's docs).

8.2.3 15.3 Anatomy of a good skill

```
---
name: r-data-viz
description: Create publication-quality ggplot2 figures following lab
  style. Use when asked to plot data, customize legends, or export
  figures for a manuscript.
---
```

````
# R Data Visualization Skill

## When to use
Use this skill whenever the user asks for a figure or plot in R.

## Steps
1. Load the data and check structure (`str()`, `head()`).
2. Choose geoms appropriate to the data type.
3. Apply the lab theme:

```r
theme_lab <- function() {
  theme_minimal(base_size = 11) +
    theme(
      legend.position = "inside",
      legend.background = element_blank()
    )
}
````

4. Save with `ggsave(..., dpi = 300)`.

8.3 Guardrails

- Never use red/green palettes alone (colorblind accessibility).
- Always set a seed before any sampling.

```
### 15.4 Adding resources

Skills can reference additional files in their directory:

- `reference.md` - background reading the agent should consult.
- `scripts/` - helper scripts the skill can invoke.
- Example inputs/outputs for regression testing.

Reference them by filename from `SKILL.md` so the agent knows they
exist.

### 15.5 Testing and iterating

1. Start with a minimal skill and one concrete task.
2. Run the task with the skill installed; observe where the agent
   misunderstands.
3. Tighten the description and instructions; add an example.
4. Iterate until the skill produces consistent results, then share it
   with your team.

### 15.6 Common pitfalls

1. Description too vague ("helps with R") - the agent never triggers it.
2. Instructions that assume context the agent does not have (write
   everything down).
3. Skills that mutate global state or delete files without asking -
   build in guardrails.

**Exercises**
```
```

1. Write a skill for one of your recurring bioinformatics tasks
   (e.g. "annotate a VCF with ANNOVAR").
2. Install it and run a realistic task; note one place the agent
   deviated from your intent and fix the skill.
3. Compare user-level vs. project-level skill placement: when is each
   appropriate?

Chapter 16: Vibe Coding with AI Agents - Lessons from a 5-Day Course

Learning objectives

- Explain what "vibe coding" means and how agents work under the hood.
- Improve prompting and collaboration with coding agents.
- Find structured learning materials to go deeper.

16.1 Background

The "AI Agents: Intensive Vibe Coding Course" (hosted by Kaggle and
Google) is a 5-day, hands-on introduction to building and working with
AI coding agents. "Vibe coding" refers to describing intent at a high
level and letting the agent produce code - powerful when done well,
chaotic when done blindly.

16.2 Under the hood: how agents actually work

Understanding the architecture demystifies capability and limits:

- **Model** - the LLM that plans and writes.
- **Tools** - file editing, shell commands, web access, search. Tools
  turn the model from a text generator into an *agent*.
- **Context / memory** - what the agent sees: conversation, files,
  repository structure.
- **Loop** - the agent iterates: act, observe results, revise.

Knowing this matters: when an agent fails, the cause is usually a tool
limitation or missing context - not "the model is dumb".

16.3 Better prompting and collaboration

Concrete practices from the course:

1. **Give the agent a role and constraints** ("you are a
   bioinformatics pipeline developer; use Nextflow DSL2; no deprecated
   modules").
2. **Provide context explicitly** - paste the error, point at the file,
   state the data format.
3. **Break large tasks into stages** and verify each stage before
   moving on.
4. **Let the agent read documentation** instead of pasting API
   details - tools like web search do this automatically.
5. **Review, don't rubber-stamp** - treat agent output as a strong
   draft that still needs your scientific judgment.

16.4 Foundations for the future

```
The course material is a launching pad for building and customizing your
own agents. Compiled resources (notebooks, materials):

- GitHub: https://github.com/fortune9/kaggle_code/tree/main/kaggle5daysofai
- Official course: https://www.kaggle.com/competitions/5-day-ai-agents-intensive-vibecoding-course-with-go

### 16.5 Common pitfalls

1. Prompting without context and expecting magic.
2. Accepting agent output without running/validating it.
3. Ignoring tool errors - read what the agent reports and give it
   feedback.

**Exercises**

1. Take one of your daily tasks and prompt an agent with a role,
   constraints, and context. Compare the output with your usual prompt.
2. When an agent fails, list the three most likely causes in terms of
   model/tools/context and check each.
3. Work through one notebook from the compiled course resources.

---

## Chapter 17: A Systematic Approach - the `zz-vibe-coding` Skill

**Learning objectives**

- Recognize the common traps of unstructured AI coding.
- Apply six principles for effective agent-driven development.
- Install a ready-made skill that encodes these principles.

### 17.1 Background: the traps

Coding with AI agents is powerful, but unstructured use hits the same
walls repeatedly:

- **Overly complex solutions** for simple tasks.
- **Endless debugging loops** when context gets bloated.
- **Missing architecture**: jumping straight into code without
  designing component boundaries.

The `zz-vibe-coding` skill distills day-to-day experience building
complex bioinformatics and software workflows into a systematic,
agent-readable methodology. (The `zz-` prefix deliberately keeps custom
user skills distinguishable from built-in ones.)

### 17.2 The six principles

1. **Architecture first** - evaluate task complexity and design
   component boundaries before writing code.
2. **Make a plan** - maintain a structured implementation plan as a
   clear roadmap during the work.
3. **Start small** - implement minimal, readable solutions without
   premature optimization.
4. **Modular design** - break functionality into focused,
```

```
   single-purpose components.
5. **Reference-driven** - align with ecosystem standards and
   authoritative documentation instead of guessing APIs.
6. **Strategic explanations** - document the strategy and intent, not
   just the code, for maintainability.

### 17.3 Installation

```bash
# Quick install via npx
npx skills add fortune9/Agent_skills --skill zz-vibe-coding

# Install globally for Claude Code specifically
npx skills add fortune9/Agent_skills --skill zz-vibe-coding --agent claude-code --global
```

Claude Code plugin marketplace:

```
/plugin marketplace add fortune9/Agent_skills
/plugin install programming@fortune9-agent-skills
```

Manual clone:

```
git clone https://github.com/fortune9/Agent_skills.git
cp -r Agent_skills/programming/zz-vibe-coding ~/.config/claude-code/skills/
```

The repository also contains skills for R, Nextflow, and AI methodologies: https://github.com/fortune9/Agent_skills

8.3.1 17.4 Adapting the principles to your team

The six principles are tool-agnostic. Teams can adopt them as code review criteria:

- Is there a plan before implementation?
- Are components single-purpose and small?
- Does the code reference authoritative docs, not forum guesses?
- Are design decisions documented as comments/ADRs?

8.3.2 17.5 Common pitfalls

1. Treating the skill as a magic wand — it improves process, it does not replace your domain knowledge.
2. Skipping the plan step for genuinely simple tasks (overhead).
3. Not customizing the skill to your team's conventions.

Exercises

1. Install the skill and run it on a small feature; observe how the agent's behavior changes (plan, small steps, references).
2. Pick a past project that went wrong and map each failure to one of the six principles.
3. Write your team's own version of the skill with your conventions.

---

9 Part VIII: Reading the Literature with AI

9.1 Chapter 18: Paper Summaries — Two Worked Examples and Critical Reading

Learning objectives

- Use AI-generated summaries as a *starting point*, not the endpoint.
- Compare two AI summaries of the same paper and identify drift.
- Apply a checklist for verifying claims against the original.
```

9.1.1 18.1 Background

AI assistants are increasingly used to summarize papers — including the posts this tutorial is based on. Two posts in this series are AI-assisted summaries of recent papers. They illustrate both the value and the danger: AI summaries are fast and readable, but they can overstate, drop nuance, or misattribute findings. Always read the original.

9.1.2 18.2 Worked example 1 — Transcriptional adaptation mediated by mRNA decay

Paper: El-Brolosy et al., *Science* (2026). https://www.science.org/doi/10.1126/science.aea1272

Summary

- **Transcriptional adaptation** is a cellular mechanism in which cells upregulate related genes in response to genetic perturbations, effectively masking the deleterious phenotypes of mutations.
- The study reveals a novel mechanism: **degraded mRNAs mediate transcriptional adaptation** by binding to antisense transcripts of related genes, which activates transcription of compensatory genes.
- The process is guided by the protein **ILF3**.

Significance: the work adds a new regulatory dimension to mRNA decay and helps explain how cells adapt to genetic perturbations.

How to verify: open the paper, find the ILF3 experiments (knockdown, binding assays), and check the antisense-transcript model in the figures before relying on the summary.

9.1.3 18.3 Worked example 2 — BRD2 clustering and transcription dynamics

Paper: *Nature Genetics* (2026). https://www.nature.com/articles/s41588-026-02533-x

Two AI agents (Gemini and Qwen) summarized this paper independently, and the summaries differ in emphasis:

Gemini variant

- BRD2 organizes transcription machinery spatially, while BRD4 mediates RNA polymerase II pause release (functional divergence).
- Histone acetylation-dependent clustering of BRD2 promotes chromatin association and dynamic cluster formation.
- Deletion of clustering domains stalls transcription, proving clustering is a functional requirement for activation.

Qwen variant

- BRD2 maintains Pol II recruitment at promoters through TFIID interaction, critical under impaired pause release.
- MOF-mediated histone H4 acetylation promotes BRD2 chromatin association and enables BRD2 clustering.
- IDR (intrinsically disordered region) deletion recapitulates the transcription machinery defects seen upon BRD2 loss.

Teaching point: both summaries describe the same study with different framings — one emphasizes functional divergence with BRD4, the other the TFIID/IDR mechanism. Neither is wrong per se, but each highlights different evidence. This is exactly why you must compare AI summaries against the paper itself.

9.1.4 18.4 A checklist for critical reading of AI summaries

1. Does the summary cite the paper (DOI)? Go read the abstract at minimum.
2. Are specific numbers/mechanisms present in the paper, or plausible inventions (hallucination)?
3. Does the summary distinguish *findings* from *interpretations*?
4. Do two summaries of the same paper agree on the key claims? Where they differ, check the original.
5. Is the "significance" claim supported by the paper's own discussion?

9.1.5 18.5 Common pitfalls

1. Copying AI summaries into manuscripts or grants without verification — a citation error or invented mechanism is embarrassing at best, disqualifying at worst.
2. Trusting the summary's framing over the paper's actual scope.

3. Ignoring the AI's own disclaimer ("generated by AI, read with a critical eye").

Exercises

1. Take any paper you know well, generate two AI summaries, and list every claim where they disagree.
2. Apply the checklist in 18.4 to one summary and report what you had to verify in the original.
3. Draft a one-paragraph policy for your lab on using AI summaries responsibly.

---

10 Appendix A: Quick Reference Cards

10.1 A.1 Shell and Bash

```
# iTerm2: set hostname for file transfers (on the remote server)
export iterm2_hostname=$(hostname -i)

# grep without killing strict-mode scripts
grep -q PATTERN file && echo found || echo "not found (ok)"
cat log | awk '/PATTERN/' | wc -l

# quoting
echo "expanded $HOME"     # expands
echo 'literal $HOME'      # literal
```

10.2 A.2 R and ggplot2

```
# Legend key size independent of data
guides(size = guide_legend(override.aes = list(size = 3)))

# Legend inside the plot
theme(legend.position = c(0.85, 0.8),
      legend.justification = c(0.5, 0.5),
      legend.background = element_blank())

# Patterns
geom_col_pattern(pattern = "stripe", pattern_density = 0.3,
                 pattern_spacing = 0.05, pattern_angle = 45)

# Regex: literal dot needs double backslash
grep("a\\.b", x)
```

10.3 A.3 Shiny and containers

```
# Pass env vars into Shiny Server workers: write them for the shiny user
RUN echo 'MY_VAR=value' >> /home/shiny/.Renviron \
    && chown shiny:shiny /home/shiny/.Renviron
```

```
# Chatbot skeleton
library(shiny); library(shinychat); library(ellmer)
ui <- fluidPage(chat_mod_ui("chat"))
server <- function(input, output, session) {
  chat_mod_server("chat", chat_openai(system_prompt = "You are a bioinformatics assistant."))
}
shinyApp(ui, server)
```

10.4 A.4 Git worktree

```
git worktree add ../hotfix -b hotfix/fix-bug
git worktree list
git worktree remove ../hotfix
```

10.5 A.5 Nextflow AWS mapping

Option	Layer	Propagates to EC2?
`aws.client.maxConnections`	Head-node JVM	No
`aws.client.multipartThreshold`	Head-node JVM	No
`aws.client.connectionTimeout`	Head-node JVM	No
`aws.client.storageClass`	Head-node JVM	Yes (via S3BashLib)
`aws.client.s3Acl`	Head-node JVM	Yes (via S3BashLib)
`aws.batch.*`	In-container S3 CLI / s5cmd	Yes (container)
Fusion config	Filesystem layer	Bypasses S3 CLI

10.6 A.6 Z-algorithm

```
def z_algorithm(s):
    n = len(s); z = [0]*n; l = r = 0
    for i in range(1, n):
        if i <= r:
            z[i] = min(r - i + 1, z[i - l])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1
        if i + z[i] - 1 > r:
            l, r = i, i + z[i] - 1
    return z

def find_pattern(P, T):
    z = z_algorithm(P + "$" + T); m = len(P)
    return [i - m - 1 for i in range(m + 1, len(z)) if z[i] == m]
```

10.7 A.7 AI agent skills

```
npx skills add posit-dev/skills --skill r-package-development -a gemini-cli
npx skills add fortune9/Agent_skills --skill zz-vibe-coding
```

---

11 Appendix B: Further Reading and Resources

11.1 Original posts this tutorial is based on

- iTerm2 shell integration: `LinkedIn/2026/0331-iterm2-file-transfer-shell-integration.md`
- Quoting rules: `LinkedIn/gemini/2026/2026-04-26-single-vs-double-quotes-in-command-line-and-r.md`
- grep exit codes: `LinkedIn/gemini/2026/2026-08-15-linux-handling-grep-exit-codes-in-scripts-with-set-e-and`
- ggplot2 legend keys: `LinkedIn/gemini/2026/2026-04-27-r-how-to-change-legend-key-size-in-ggplot2.md`
- ggpattern: `LinkedIn/gemini/2026/2026-05-22-r-how-to-use-ggpattern.md`
- Legend inside plot: `LinkedIn/gemini/2026/2026-05-30-r-position-legend-inside-plot.md`
- shinychat + ellmer: `LinkedIn/gemini/2026/2026-06-06-build-ai-chatbot-with-shinychat-and-ellmer.md`
- Shiny Server env vars: `LinkedIn/gemini/2026/2026-08-15-r-environment-variables-in-r-shiny-server-container`
- Git worktree: `LinkedIn/gemini/2026/2026-06-27-git-worktree-working-on-multiple-branches-simultaneously.m`

- blogdown themes: `LinkedIn/gemini/2026/2026-04-18-modify-blogdown-theme.md`
- Nextflow AWS: `LinkedIn/gemini/2026/2026-07-25-nextflow-aws-options-where-they-take-effect.md`
- pdf.db.builder: `LinkedIn/gemini/2026/2026-07-11-pdf-db-builder.md`
- Z-algorithm: `LinkedIn/gemini/2026/2026-05-22-z-algorithm-for-string-matching.md`
- Agent skills intro: `LinkedIn/2026/0307-make-AI-agent-smarter-with-skills.md`
- Gemini CLI skills: `LinkedIn/gemini/2026/2026-04-18-creating-custom-agent-skills-with-gemini-cli.md`
- Vibe coding course: `LinkedIn/gemini/2026/2026-06-22-ai-agents-intensive-vibe-coding-course.md`
- zz-vibe-coding: `LinkedIn/gemini/2026/2026-08-08-zz-vibe-coding-agent-skill.md`
- Paper summary (mRNA decay): `LinkedIn/2026/0213-transcriptional-adaptation-by-degraded-RNAs.md`
- Paper summary (BRD2): `LinkedIn/gemini/2026/0412-histone-acetylation-dependent-clustering-of-brd2-instr` and `LinkedIn/qwen/2026/0412-histone-acetylation-dependent-clustering-of-brd2-instructs-transcription-d`

11.2 Blog articles (full versions)

- https://fortune9.netlify.app/2026/04/18/creating-custom-agent-skills-with-gemini-cli/
- https://fortune9.netlify.app/2026/04/18/r-how-to-modify-the-theme-used-by-blogdown/
- https://fortune9.netlify.app/2026/04/26/single-vs-double-quotes-in-command-line-and-r/
- https://fortune9.netlify.app/2026/04/27/r-how-to-change-legend-key-size-in-ggplot2/
- https://fortune9.netlify.app/2026/05/22/r-how-to-use-ggpattern/
- https://fortune9.netlify.app/2026/05/22/z-algorithm-for-string-matching/
- https://fortune9.netlify.app/2026/05/30/r-position-legend-inside-plot/
- https://fortune9.netlify.app/2026/06/27/git-worktree-working-on-multiple-branches-simultaneously/
- https://fortune9.netlify.app/2026/07/25/nextflow-aws-options-where-they-take-effect/
- https://fortune9.netlify.app/2026/08/15/linux-handling-grep-exit-codes-in-scripts-with-set-e-and-set-o-pipefail/
- https://fortune9.netlify.app/2026/08/15/r-environment-variables-in-r-shiny-server-container/

11.3 Repositories

- https://github.com/fortune9/Agent_skills — agent skills (zz-vibe-coding, R, Nextflow)
- https://github.com/fortune9/pdf_db_builder — searchable PDF database
- https://github.com/fortune9/sample-programs — sample programs (shiny_chatbot.R)
- https://github.com/fortune9/kaggle_code — Kaggle 5-day AI course materials
- https://github.com/fortune9/programming_learning_notes — agent-skills guide

11.4 Official documentation

- Agent skills (VS Code / Copilot): https://code.visualstudio.com/docs/copilot/customization/agent-skills
- Gemini CLI: https://geminicli.com
- skills.sh: https://skills.sh
- shinychat: https://rstudio.github.io/shinychat/
- ellmer: https://ellmer.tidyverse.org/
- Nextflow AWS Batch: https://www.nextflow.io/docs/latest/aws.html
- GROBID: https://grobid.readthedocs.io/